

The binary numerical system by Applicative functors

Luca Roversi

December 2, 2025

Learning outcomes

- Binary operations on a functor for Binary Numbers
- The Binary Numbers are a Numerical System
- Exercises

A numerical system requires binary operations

-- The class Num to build a numerical system

```
type Num :: * -> Constraint
```

```
class Num a where
```

```
    (+) :: a -> a -> a
```

```
    (-) :: a -> a -> a
```

```
    (*) :: a -> a -> a
```

```
    negate :: a -> a
```

```
    abs :: a -> a
```

```
    signum :: a -> a
```

```
    fromInteger :: Integer -> a
```

Starting point.

Numbers.Positive.OperationsBinary.hs

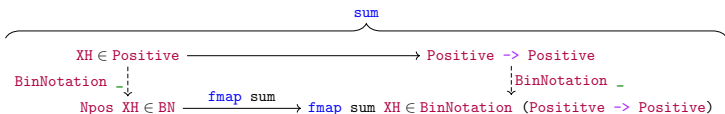
A relevant question

**“Can we obtain binary operations on `BN`
by lifting those ones on `Positive`?”**

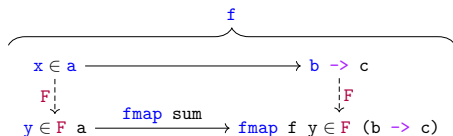
The problem

`sum :: Positive -> Positive -> Positive`

$\Leftrightarrow \text{sum} \in \text{Hom}(\text{Positive}, \text{Positive} \rightarrow \text{Positive})$



It is a general problem.



Applicative Functors in Haskell

Applicative Functor (pattern) $\Leftrightarrow$ (`f (a->b)` , `pure` , `(<*>)`) + axioms

```
ghci> :i Applicative
type Applicative :: (* -> *) -> Constraint
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a->b) -> f a -> f b
  GHC.Base.liftA2 :: (a -> b -> c) -> f a -> f b -> f c
  {-# MINIMAL pure, ((<*>) / liftA2) #-}
```

Axioms.

- *Identity.* `pure id <*> v = v`
- *Composition.* `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`
- *Homomorphism.* `pure f <*> pure x = pure (f x)`
- *Interchange.* `u <*> pure y = pure ($ y) <*> u`

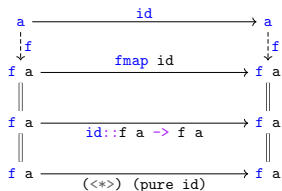
Meaning and Typing of the Applicative laws

Identity `pure id <*> v = v`

Meaning.

- `pure id` does “nothing”, exactly like `id`
- base to ensure that an applicative functor can generate a category

Associated diagram.



- Being `f` a functor justifies the 2 topmost squares
- Being `f` a functor let the 2nd topmost square commute
- The two identities
 - `fmap id x = x`
 - `pure id <*> x = x`

let the lowermost square commute

Composition $((\text{pure}(\cdot) \lt * \gt u) \lt * \gt v) \lt * \gt w = u \lt * \gt (v \lt * \gt w)$

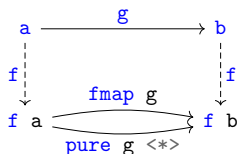
Meaning.

- $\text{pure}(\cdot) \lt * \gt u \lt * \gt v$ is a *composition* analogous to $(\cdot)$
- the axiom is associativity

Type checking.

$$\begin{array}{c}
 \text{apply } \lt * \gt \frac{u :: f \ (b \rightarrow c) \quad \text{apply } \lt * \gt \frac{v :: f \ (a \rightarrow b) \quad w :: f \ a}{v \lt * \gt w :: f \ b}}{u \lt * \gt (v \lt * \gt w) :: f \ c} \\
 \\
 \text{apply } \lt * \gt \frac{\text{pure } (\cdot) :: f \ ((b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow c) \quad u :: f \ (b \rightarrow c)}{\text{pure } (\cdot) \lt * \gt u :: f \ ((a \rightarrow b) \rightarrow a \rightarrow c)} \quad \frac{v :: f \ (a \rightarrow b)}{(\text{pure } (\cdot) \lt * \gt u) \lt * \gt v :: f \ (a \rightarrow c)} \quad w :: f \ a \\
 \hline
 ((\text{pure } (\cdot) \lt * \gt u) \lt * \gt v) \lt * \gt w :: f \ c
 \end{array}$$

fmap g vs pure g vs f(a->b)



$$g :: a \rightarrow b \xrightarrow{\text{pure}} f(a \rightarrow b)$$

$$(f(a \rightarrow b), f a) \xrightarrow{\text{uncurry } (<*>)} f b$$

Key distinctions:

- `fmap g`:
 - *lifts* `g`: it takes “`f a`” producing “`f b`”
 - is an arrow in “`f a -> f b`”
- `pure g`:
 - *embeds* `g` into a context
 - is *an element* of `f(a->b)`
- `f(a->b)`:
 - can contain “`pure`” functions
 - can contain “effectful” functions from composition
`(<*>) ((<*>) (pure (.) u)) v`
 - its elements can act on `f a` only via `(<*>)`
- `fmap g = (<*>) (pure g)`

Homomorphism $\text{pure } g \text{ <*> pure } x = \text{pure } (g \ x)$

Meaning.

- “pure” function applied by <*> to a “pure” value is the same as applying as usual the function to the value, then using **pure**
- “**pure** (and <*>) preserves usual function application”

Type checking the terms.

$$\text{apply pure} \frac{g :: a \rightarrow b}{\text{pure } g :: f \ (a \rightarrow b)} \quad \text{apply pure} \frac{x :: a}{\text{pure } x :: f \ a} \\ \text{apply <*>} \frac{\text{pure } g :: f \ (a \rightarrow b) \quad \text{pure } x :: f \ a}{\text{pure } g \text{ <*> pure } x :: f \ b}$$

$$\text{application} \frac{g :: a \rightarrow b \quad x :: a}{g \ x :: f \ b} \\ \text{apply pure} \frac{g \ x :: f \ b}{\text{pure } g \ x :: f \ b}$$

Interchange $u \text{ <*> pure } y = pure ((\$) y) \text{ <*> } u$

Meaning.

- recall that $(\$ y)$ is the function that supplies y as an argument to another function
- applying a morphism to a “pure” value $pure y$ is the same as applying $pure (\$ y)$ to the morphism

Type checking the terms. Let $(\$) :: (a \rightarrow b) \rightarrow a \rightarrow b$ be given.

$$\begin{array}{c}
 \text{build a section of } \$ \frac{(\$) :: (a \rightarrow b) \rightarrow a \rightarrow b \quad y :: a}{(\$ y) :: (a \rightarrow b) \rightarrow b} \\
 \text{apply } pure \frac{(\$ y) :: (a \rightarrow b) \rightarrow b}{pure (\$ y) :: f ((a \rightarrow b) \rightarrow b)} \quad u :: f (a \rightarrow b) \\
 \text{apply } <*> \frac{pure (\$ y) :: f ((a \rightarrow b) \rightarrow b) \quad u :: f (a \rightarrow b)}{pure (\$ y) <*> u :: f b}
 \end{array}$$

$$\begin{array}{c}
 \text{apply } <*> \frac{u :: f (a \rightarrow b) \quad \frac{y :: a}{pure y :: f a}}{u <*> pure y :: f b}
 \end{array}$$